

BASTION LINE

Rapport Technique Officiel — Prochaine version candidate bêta

Trajectoires ennemies plus propres dans les virages, tour d'archers à socle fixe + baliste orientable (asset Leonardo), réévaluation de la taille des tours sur téléphone

Date du rapport	6 octobre 2026
Dépôt	github.com/scuillerjack-oss/bastion-line-v0
État réel de reprise (audit court, cahier section 1)	fa74138 (archer : socle fixe + baliste orientable déjà livré avant cette session) -- travail conservé, jamais de retour arrière
Branche	main
Commit final	144451c20e2b88f407f6124d254d31a9faeeb4f5
Identifiant de build vérifié	v0.1.0+144451c
HEAD local = HEAD distant	Vérifié (git fetch + git rev-parse, 0 commit d'écart dans les deux sens, et confirmé via l'API GitHub list_commits)
Statut du déploiement (job-level)	Workflow « Deploy to GitHub Pages » #42 (run 37409001907) : job build -- npm test (372/372), npm run build, Playwright + npm run test:mobile (35/35) -- succès ; job deploy (actions/deploy-pages@v4) -- succès
URL publique (bêta téléphone)	https://scuillerjack-oss.github.io/bastion-line-v0/
Statut de la mission	Candidate à une NOUVELLE bêta physique humaine -- NON déclarée gelée ni finalisée

0. Audit court de reprise

Conformément à la règle absolue du cahier (« ne reconstruis pas, ne recommence pas de zéro, ne reviens pas à une ancienne version »), la mission a repris strictement l'état réel final du dépôt, sans aucune refonte. État retrouvé au lancement de cette session : branche `main`, HEAD local = HEAD distant = `fa74138` (« socle fixe + baliste orientable (tour d'archers) », déjà livré lors d'une phase antérieure de cette même mission), arbre de travail propre, 313/313 tests, 35/35 vérifications mobiles, déploiement vérifié à ce commit. Les assets Leonardo (carte, canon, tour d'archers combinée, baliste) et les scripts de traitement associés étaient tous présents et fonctionnels. Aucun travail non commité, aucun blocage.

1. Priorité 1 -- Trajectoires ennemies

1.1 Diagnostic

La route `PATH_MAP` (71 points, simplification Douglas-Peucker $\epsilon=1,5$ sur le tracé dense centré de référence, `engine/levels.js`) contenait des changements de direction INSTANTANÉS jusqu'à $67,3^\circ$ à certains sommets (moyenne $27,7^\circ$ sur les 69 sommets intérieurs, 41 au-delà de 20°), mesurés directement sur les données live du jeu. Douglas-Peucker ne borne que l'écart perpendiculaire au tracé simplifié (1,5 unité) -- jamais l'angle de virage résultant : un virage réel et serré de la route (confirmé visuellement : un tronçon quasi horizontal suivi d'un tronçon quasi vertical sur une très courte portée) se traduisait par un unique sommet à angle vif, perçu par un ennemi en interpolation linéaire pure (`engine/path.js`, INCHANGÉ comme demandé) comme un pivot sur place -- exactement le symptôme « pas encore assez propre dans les virages » remonté par la bêta humaine.

1.2 Remède

`scripts/smooth_path_turns.py` (nouveau, self-contenu, idempotent) : une coupe de coin ciblée (même principe que l'algorithme de Chaikin), appliquée UNIQUEMENT aux sommets dont l'angle de virage dépasse 12° , remplaçant chaque sommet vif par deux sommets plus doux reliés par une courte corde (30% de chaque segment adjacent), répétée par passes jusqu'à stabilisation. `engine/path.js` n'a reçu AUCUNE modification -- c'est purement une amélioration des DONNÉES du chemin, conformément à l'instruction du cahier de « continuer dans cette direction, jamais remplacer la logique ».

Mesure	Avant	Après
Angle de virage maximal	$67,3^\circ$	$19,7^\circ$
Angle de virage moyen	$27,7^\circ$	$7,5^\circ$
Sommets $> 20^\circ$	41 / 69	0 / 205
Nombre de points <code>PATH_MAP</code>	71	209
Écart maximal au corridor réel mesuré (tolérance test : 6, demi-largeur route : 17)	n/a	2,91 unités

Vérifié : la nouvelle polyligne reste très en-deçà du corridor réel de la route sur toute sa longueur (`tests/pathing.test.js`, inchangé dans sa méthode), et un nouveau test verrouille désormais l'angle de virage maximal pour toute régression future, sur les 50 niveaux (`tests/pathing.test.js`). Vérifié visuellement par captures d'écran pendant une vague réelle à 5 ennemis (Niveau 1) : mouvement visiblement fluide et centré dans les virages, aucun saut observé.



5 ennemis en vague réelle (niveau 1) traversant le premier virage après lissage -- mouvement groupé et centré, sans saut.

2. Priorité 2 -- Tour fixe + baliste orientable (test technique principal)

Livré lors de la phase précédente de cette même mission (commit fa74138), avant la reprise de cette session -- conservé intact, revérifié ici.

2.1 Nouvelle architecture à deux couches

- **Socle** (scripts/process_archer_socle.py) : recadrage du sprite combiné existant en retirant le mécanisme d'arbalète (coupure à $y=195$, la plus haute ligne totalement propre, vérifiée visuellement sur plusieurs candidats). JAMAIS retourné ni pivoté -- le code qui le dessine ne lit même pas `tower.aimAngle`.
- **Baliste** (scripts/process_leonardo_baliste.py) : 8 sprites directionnels découpés de la planche Leonardo fournie (grille 3x3, centre vide -- la position dans la grille encode directement la direction visuelle du carreau, vérifiée cellule par cellule). Fond « transparent » retiré par flood-fill depuis les bords de l'image entière (la planche est un .jpg : le damier est un pixel opaque baked, jamais un vrai canal alpha -- un simple seuil de couleur global aurait pu ronger un reflet clair à l'intérieur du mécanisme). Pivot (centre du plateau rotatif) mesuré individuellement par orientation et vérifié par une composition de test (les 8 sprites, dessinés chacun à son propre pivot sur un même point d'ancrage, alignent leurs 8 plateaux en un seul anneau cohérent).

2.2 Sélection directionnelle + hystérésis

engine/simulation.js, `stepTowerSpriteDirection()` : calculée en continu à partir de `tower.aimAngle` déjà lissé (jamais recalculée indépendamment) -- sélectionne l'une des 8 directions compas. Une hystérésis dédiée (secteur « collant » de $28,5^\circ$, jamais $22,5^\circ$ strict) empêche un clignotement du sprite quand la cible oscille près d'une frontière angulaire, sans jamais devenir un blocage

permanent (vérifié : au-delà de la marge, la direction bascule normalement).

2.3 Rendu avec dégradation en 3 paliers

ui/render.js, drawArcherTower () : socle+baliste (nouveau système) -> ancien sprite combiné (repli intermédiaire) -> silhouette Canvas (repli final). Jamais d'écran cassé pendant un chargement ou en cas d'échec réseau -- même discipline que le reste du projet.



Tour d'archers en jeu : socle fixe + baliste orientable, vue rapprochée réelle (niveau 3, 390x844).

3. Taille des tours sur mobile

Archer : déjà traité lors de la livraison du socle/baliste (54x46, +12% vs les 48px hérités de V7). Canon : échelle V7 (1,52) déjà adéquate, confirmée par capture d'écran mobile directe. Catapulte (placeholder Canvas, asset Leonardo non fourni cette mission -- toujours hors périmètre pour ses orientations propres) : une capture d'écran directe (niveau 3, 390x844, les trois tours côte à côte) a montré sa silhouette nettement plus frêle que les deux autres tours, en retrait de présence visuelle. Remède : même hausse « légère » que la tour d'archers en V7 (1,28 -> 1,4, ui/render.js) -- identique forme vectorielle, simplement plus grande, jamais un changement de visuel ou une nouvelle orientation (qui reste explicitement hors périmètre).



Les trois familles de tours sur la même carte (niveau 3, 390x844) : archer et catapulte construits, canon visible dans le panneau de construction.

4. Unités ennemies -- vérification de périmètre

Vérifié l'état réel de la tâche précédente concernée (V7, « intégrer les soldats Leonardo ») : elle avait bien été explicitement demandée, mais reste BLOQUÉE faute d'asset -- ENEMY_SPRITE_CONFIG (ui/render.js) est toujours délibérément vide, le pipeline étant prêt mais sans asset Leonardo d'ennemi jamais fourni. Cette mission n'a apporté aucune planche Leonardo pour les ennemis (uniquement la baliste). Conformément au cahier (« si elle ne faisait pas partie du périmètre précédent, ne lance pas une refonte lourde non demandée ») et à son instruction explicite de ne pas retarder le test prioritaire, les graphismes d'ennemis n'ont PAS été retouchés cette mission.

5. Tests obligatoires (cahier, section 8)

5.1 Suite automatisée

372/372 tests (node:test), incluant 2 nouveaux tests de non-régression pour le lissage de trajectoire (angle de virage borné sur les 50 niveaux) et 1 nouveau test de charge réelle pour la baliste.

5.2 Test obligatoire dédié -- plusieurs ennemis traversant rapidement la portée

tests/simulation.test.js : 8 ennemis « Éclaireur » (vitesse 125, la plus rapide des 4 archétypes) en train serré sur un chemin de test, tour décalée de l'axe pour balayer un véritable arc d'angles, ~260 ticks de simulation réelle (le vrai moteur de ciblage `findTarget` remplaçant continuellement la cible). Vérifié à CHAQUE tick :

- position du socle (`tower.x/tower.y`) strictement invariante -- 0 violation sur l'ensemble du test ;

- chaque direction choisie reste un index valide des 8 compas (jamais NaN/undefined/incohérent) ;
- aucun retour A->B->A à moins de 5 ticks d'écart (vibration permanente) -- l'hystérésis, déjà verrouillée sur un angle forcé synthétique par deux tests antérieurs, tient sous une charge réelle à plusieurs ennemis.

5.3 Autres points de la liste obligatoire

Point du cahier	Couverture
Placement / acquisition / changement de cible / tir	Tests unitaires existants (simulation.test.js) + check-mobile.mjs (cycle construire/tap réel)
Stabilité absolue du socle	Nouveau test §5.2 + socle ne lit même pas aimAngle (code)
Sprites directionnels / pivot	process_leonardo_baliste.py (composition de test d'alignement) + vérification visuelle §2.3
Plusieurs ennemis / virages	Nouveau test §5.2 + interpolation.test.js (virage à angle droit) + captures §1.2
Centrage routes / fluidité	pathing.test.js (corridor + angle de virage, 50 niveaux)
Upgrades / vente / changement de niveau / reprise	check-mobile.mjs (35/35, inchangé)
Audio / mobile / PWA	check-mobile.mjs (35/35, build ID vérifié v0.1.0+144451c)

6. Fichiers et assets modifiés cette session

- **Nouveau** scripts/smooth_path_turns.py -- lissage des virages de PATH_MAP.
- **Modifié** src/engine/levels.js -- nouveau PATH_MAP (209 points, ancrages manuels d'entrée/sortie conservés).
- **Modifié** src/ui/render.js -- FALLBACK_SCALE_BY_FAMILY.longue_portee (1,28 -> 1,4).
- **Modifié** tests/pathing.test.js -- nouveau verrou d'angle de virage maximal (50 niveaux).
- **Modifié** tests/simulation.test.js -- nouveau test de charge réelle multi-ennemis sur la baliste.

(Rappel : scripts/process_archer_socle.py, scripts/process_leonardo_baliste.py, les 8 sprites public/assets/towers/baliste/*.png et public/assets/towers/archer_socle.png avaient déjà été livrés avant cette session, commit fa74138 -- non retouchés ici.)

7. Anomalies restantes

- Catapulte : toujours un placeholder Canvas (aucun asset Leonardo fourni) -- hors périmètre explicite de cette mission.
- Ennemis : toujours la silhouette Canvas provisoire -- bloqué faute d'asset Leonardo (voir section 4), non retardé intentionnellement.
- Vérification de la page publique servie : la politique réseau de cet environnement d'exécution bloque les requêtes HTTPS sortantes directes vers github.io -- la vérification du déploiement s'appuie donc sur les logs bruts job-level de GitHub Actions (build ET deploy en succès pour le commit exact 144451c), la même méthode que pour tous les rapports précédents de ce projet.

8. Critères de réussite -- bilan

Critère (cahier)	Statut
Ennemis suivent encore mieux les routes	Oui -- angle de virage max 67,3° -> 19,7°, verrouillé par test
Mouvements plus fluides	Oui -- vérifié visuellement (vague réelle, captures)
Gameplay satisfaisant préservé	Oui -- 0 régression, 372/372 tests
Corps de la tour d'archers parfaitement fixe	Oui -- ne lit jamais aimAngle, verrouillé par test de charge réelle
Seule la baliste change d'orientation	Oui
Plus de saut de toute la tour au changement de direction	Oui -- architecture 2 couches
Asset Leonardo intégré proprement	Oui -- alpha propre, pivot vérifié, dimensions cohérentes
Tours suffisamment visibles sur téléphone	Oui -- archer et catapulte réévalués, canon déjà adéquat
Tests passent	372/372 + 35/35 mobile, CI job-level vert au commit final
Aucune régression majeure	Oui

9. Confirmation finale

Commit final 144451c20e2b88f407f6124d254d31a9faeeb4f5, branche main. HEAD local = HEAD distant (vérifié par `git fetch` et par l'API GitHub). CI job-level verte pour ce commit exact (build : tests+mobile ; deploy : actions/deploy-pages@v4). La candidate bêta est accessible publiquement à <https://scuillerjack-oss.github.io/bastion-line-v0/>, en PWA installable, et n'est PAS déclarée comme version finale -- elle attend la prochaine bêta humaine pour valider en particulier la méthode socle fixe + baliste orientable avant extension au canon et à la catapulte.